

LPFY

How LPFY Markets work

Isolated per-pair ERC-4626 vaults hold idle USDC. Borrowers post Uniswap LP NFTs and borrow under an LTV cap. Liquidations repay the vault, then split surplus between `feeTo` and the borrower. Pricing uses Chainlink; liquidations can run via CRE.

1. Overview

LENDERS

ERC-4626 PairVault
One vault per pair. Supply with `deposit` / `redeem`. Idle USDC sits in the vault, not on the market.

BORROWERS

LP NFT collateral
Lock a Uniswap V3/V4 LP NFT and borrow Circle USDC from the matching pair under LTV limits.

FEES

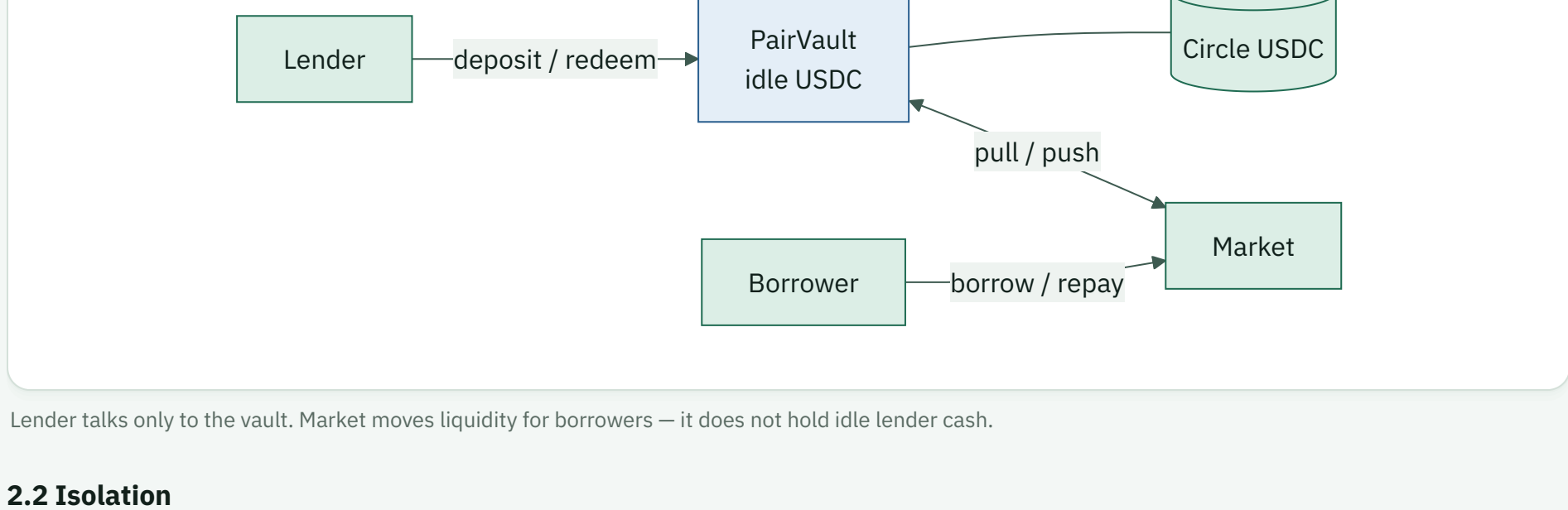
Surplus on liquidate
`LiquidationFeeBps` of post-debt surplus → `feeTo` (separate from Ownable owner). Rest → borrower.

Lenders supply through `PairVault.deposit`. The market does not take direct lender deposits — it only pulls and pushes vault liquidity for borrows and repayments.

2. ERC-4626 PairVaults

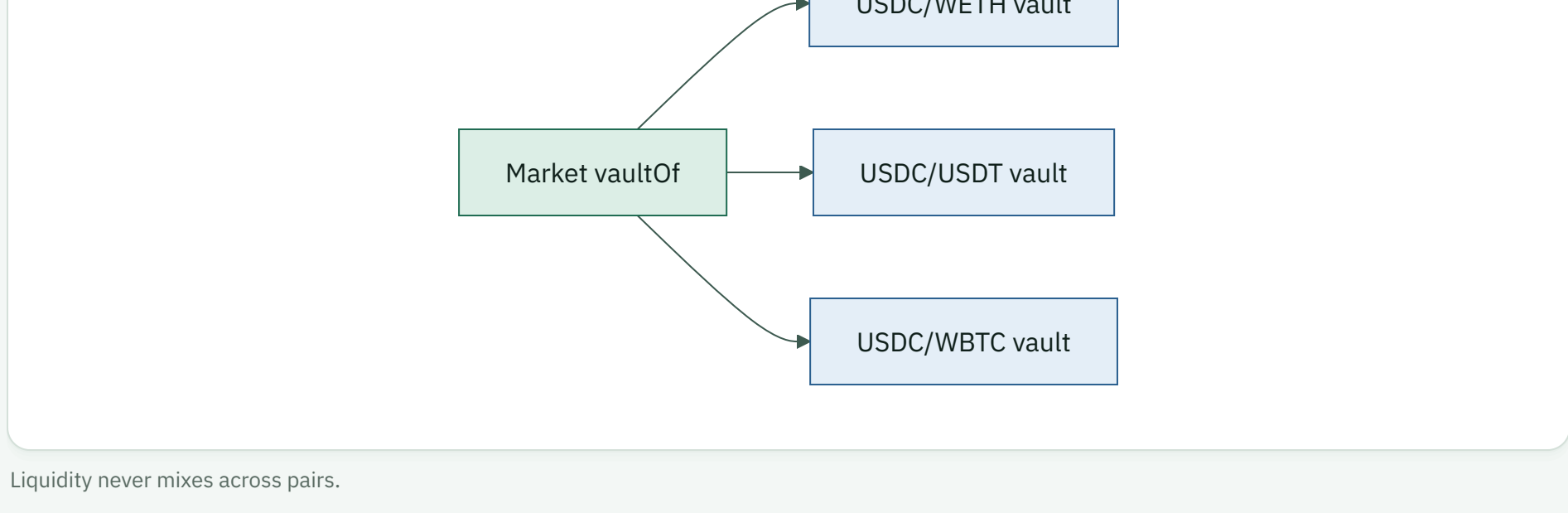
Each pair has an isolated book on `MarketLendingModule` and a dedicated **PairVault** holding idle Circle USDC.

2.1 Money flow



Lender talks only to the vault. Market moves liquidity for borrowers — it does not hold idle lender cash.

2.2 Isolation



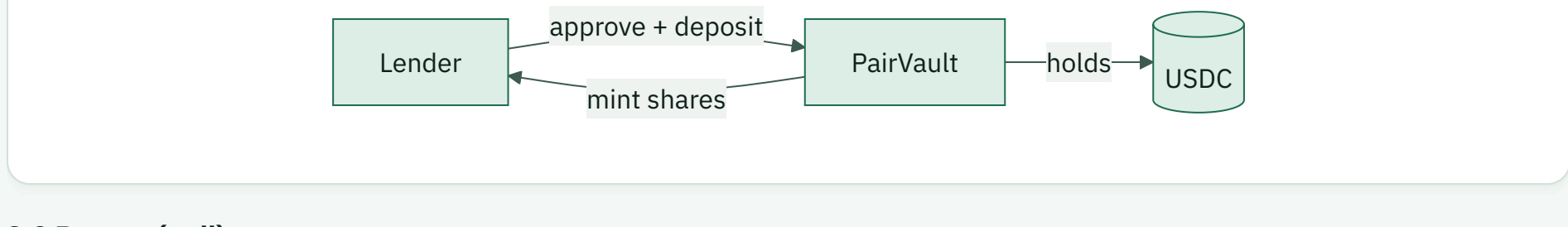
Liquidity never mixes across pairs.

STATE	WHERE USDC SITS	COUNTED IN
Idle	In the PairVault	<code>idleAssets()</code> < <code>totalAssets()</code>
Lent out	Borrower wallets	Market principal (+ interest book)
After repay / liquidate	Pushed back to vault	Idle cash rises; lenders can redeem

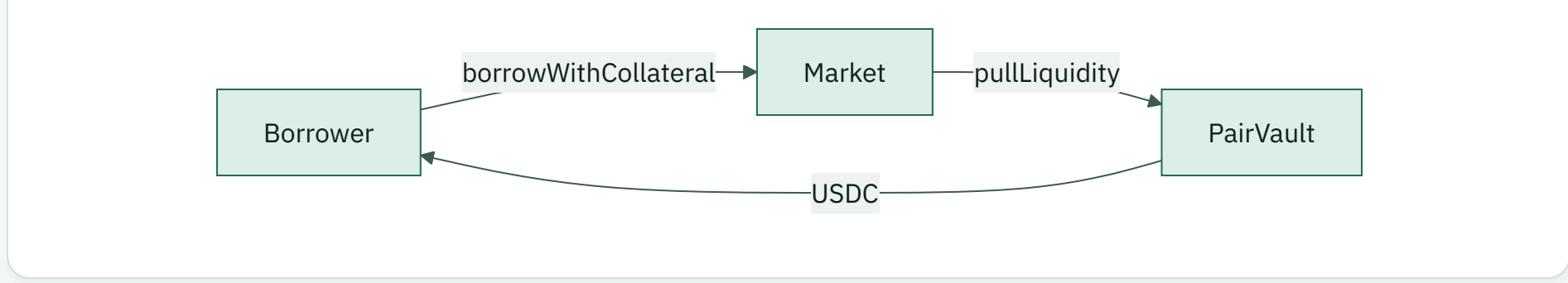
3. End-to-end flows

Same ideas as the money map — one short path per action.

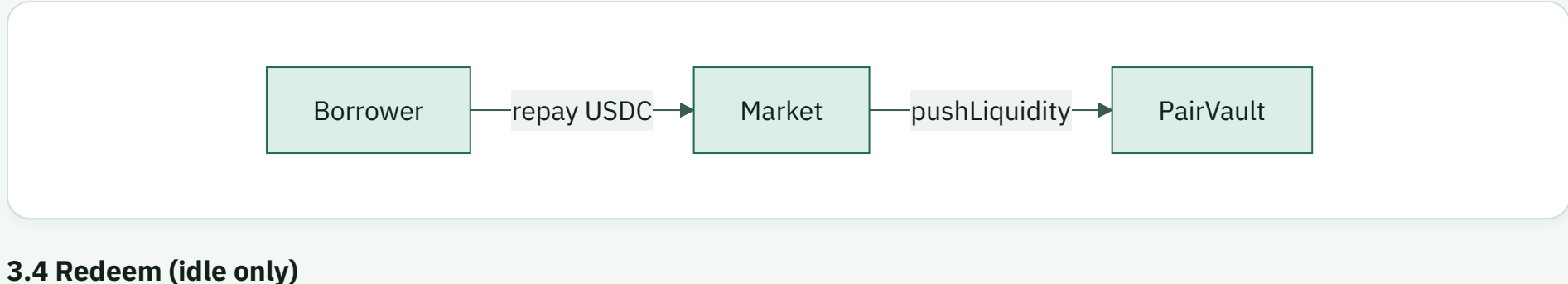
3.1 Deposit



3.2 Borrow (pull)



3.3 Repay (push)



3.4 Redeem (idle only)

`maxWithdraw` / `maxRedeem` are capped by idle vault cash. High utilization can block full exit until repay / liquidation returns USDC.

4. Share pricing

Share price = `totalAssets` / `totalSupply`. PairVault overrides `totalAssets` so lenders accrue when borrowers owe interest — not only when cash sits idle.

```
totalAssets() = idleAssets() + assets owed by the market
               = USDC idle in vault
               + totalPrincipal + totalAccruedInterest
```

Accrued interest raises share *value*. Converting that value to USDC still needs cash returned via repay or liquidation.

5. Liquidation fee structure

Fees apply only on successful liquidation when recovery exceeds debt. They are configured on the market, not on CRE or the vault.

VARIABLE	MEANING
<code>liquidationFeeBps</code>	% of surplus (after vault debt repaid), in basis points. $1000 = \mathbf{10\%}$
<code>feeTo</code>	Wallet that receives the protocol fee — independent of Ownable owner
Borrower surplus	Remaining surplus after the fee
Shortfall	If recovery < debt → no fee, no surplus to borrower

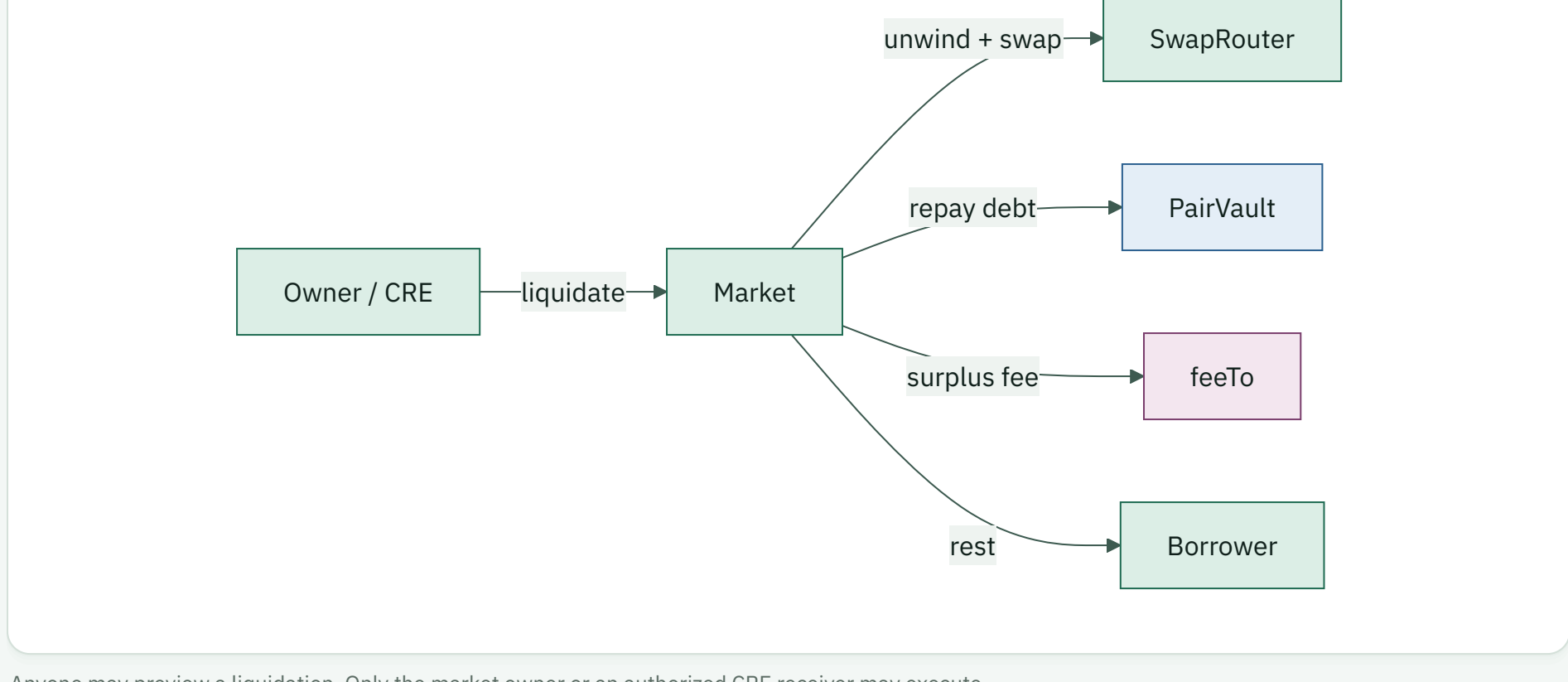
Admin (owner only): `setLiquidationFeeBps` · `setFeeTo`

Surplus split

```
debt repaid to PairVault = principal + interest
surplus = recoveryUSDC - debt
fee = surplus × liquidationFeeBps / 10000 → feeTo
borrower gets = surplus - fee
```

6. Liquidation path

- Loan is underwater (debt / collateral ≥ liquidation threshold).
- Owner or CRE liquidation receiver calls `liquidate`.
- Adapter unwinds LP NFT → collect tokens (empty NFT → borrower).
- Non-USDC legs are swapped to Circle USDC.
- Debt is pushed back into the pair’s PairVault.
- `liquidationFeeBps` of remaining surplus → **feeTo**.
- Leftover surplus → borrower; loan closed.



Anyone may preview a liquidation. Only the market owner or an authorized CRE receiver may execute.

CALLER	ALLOWED?
Market owner	Yes
CRE liquidation receiver	Yes
Anyone else	No

7. Who does what

LENDER

Deposits and redeems USDC via the pair vault. Earns from borrower interest through share price.

BORROWER

Posts LP NFT collateral, borrows USDC, repays debt, and withdraws the NFT when debt is zero.

OWNER

Configures risk params, fees, and may liquidate. Liquidate and admin actions are owner-only in the app.

CRE

Automated keeper that detects underwater loans and triggers liquidate. Does not set fees.

FEETO

Receives the protocol cut of liquidation surplus when recovery exceeds debt.

ORACLES

Chainlink Data Feeds price collateral for LTV and liquidation health checks.

8. References

GIT

Repository
github.com/unblocktechie/LPFy
Key packages: `smart-contracts/` · `web/` · `cre/` · `docs/`

FILES

Where to look in the repo

PATH	ROLE
<code>smart-contracts/contracts/markets/PairVault.sol</code>	ERC-4626 idle USDC vault
<code>smart-contracts/contracts/markets/MarketLendingModule.sol</code>	Loans, LTV, liquidate, fees
<code>smart-contracts/contracts/markets/CreLiquidationReceiver.sol</code>	CRE → liquidate
<code>smart-contracts/deployments/sepolia-markets.json</code>	Live Sepolia addresses + <code>webEnv</code>
<code>cre/liquidation-keeper/workflow.ts</code>	CRE cron keeper logic
<code>web/src/features/supply/SupplyPanel.tsx</code>	Vault <code>deposit</code> / <code>redeem</code> UI

Quick recap

- Isolated pools:** each pair has its own books and PairVault.
- ERC-4626:** lenders use standard `deposit` / `mint` / `withdraw` / `redeem`.
- LPFy:** idle USDC in vault; market pulls on borrow, pushes on repay / liquidate.
- Costody:** `totalAssets` = `idle` + `principal` + `accrued interest`.
- Fees:** after vault is repaid, `liquidationFeeBps` of surplus → `feeTo`; rest → borrower.
- Authority:** liquidate is owner or CRE only.

Deployed contracts (Sepolia)

Canonical file: `smart-contracts/deployments/sepolia-markets.json`. Full setup: [SETUP AND DEPLOY.md](#).

CONTRACT	ADDRESS
<code>MarketLendingModule</code>	<code>0x658A244Ad0c51F0c49d1fF79678bc642365b6E65</code>
<code>ValuationOracle</code>	<code>0xA659A2C34c5E9026f777AF654d0F742f8444f7F</code>
<code>V3Adapter</code>	<code>0x8Aafb1e3941E9b5f6b947e6213ab68f397845768</code>
<code>V4Adapter</code>	<code>0x3234ac1508034A9494baa3d82D055a6b1ae928d</code>
<code>BorrowRateConfig</code>	<code>0xb2e0c7b38b7DECE87A4E3D41A465F0175Ff65d25</code>
<code>StaticApysource</code>	<code>0x2b0869EB3168CBaCc7167Dc1764637a7e8D296C2</code>
<code>CreLiquidationReceiver</code>	<code>0xc9682A8649B7587e43a9cb85068d23081C5397F7</code>
<code>PairVault USDC/WETH</code>	<code>0x9478AD78af6c7758ADf8Aa78C2cc5ADEd5990843</code>
<code>PairVault USDC/USDT</code>	<code>0xC934C71ed024Da403f868EC4C9e26C996d7E7ca</code>
<code>PairVault USDC/WBTC</code>	<code>0xab30B8fEBEAdf0B2094Db087Df3CA4E632e27Ca0</code>